파인튜닝을 통한 KPoEM 모델 구현

이 장에서는 구축한 KPoEM 데이터셋을 활용해 인문학 연구, 그중에서 한국 근대시 감정 분석에 특화된 딥러닝 모델 구현의 실제 과정을 살펴보도록 한다. 파인튜닝에 활용된 코드 역시 링크를 통해 공개해 두었으므로, 직접 다운로드하여 살펴볼 수 있게 하였다.

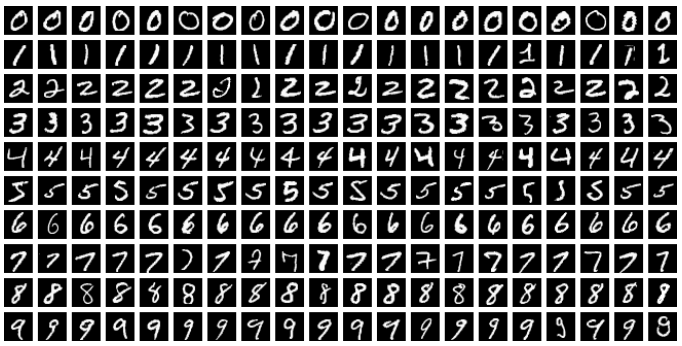
1) 인문 데이터와 파인튜닝

앞서 KPoEM 데이터셋 구축 과정을 통해, 인문 데이터의 실례를 살펴보았다. 이러한 데이터셋 구축은 필연적으로 많은 인력과 시간이 들기 때문에 쉬이 착수하기 어려운 것이 현실이다. 그럼에도 불구하고 이러한 인문 데이터셋을 만든다면, 그것을 어떻게 활용할 수 있을까?

이 절에서는 인문 데이터의 활용, 특히 딥러닝 모델 구현과 관련된 내용을 살펴보도록 한다.

인문 레이블 데이터

2장에서 살펴본 것처럼, 주석 처리한 인문 레이블 데이터는 딥러닝 모델 구현에 활용될 수 있다. 통상적으로 데이터셋을 딥러닝 모델 학습에 사용한다고 했을 때, 가장 먼저 떠올릴 만한 데이터셋의 사례로 다음의 MNIST(Modified National Institute of Standards and Technology) 데이터셋을 거론해 볼 수 있다.



MNIST 데이터셋은 아라비아 숫자 손글씨 이미지로 구성된 데이터셋으로, 60,000개의 훈련 데이터셋, 10,000개의 테스트 데이터셋으로 구성되어 있다. 1998년 발표된 이 데이터셋은 기계 학습, 그중에서도 딥러닝과 관련해서는 사실상 표준적인 데이터셋으로 자리매김하여 오늘날 딥러닝을 실습하는 사람이라면 누구나 한 번쯤 거치는 유명한 데이터셋이 되었다. 그렇다면 MNIST 데이터셋은 인문 데이터라고 할 수 있을까?

물론 이러한 데이터셋은 인문 데이터와는 거리가 멀다. 인문 데이터라 함은, 객체 탐지(object detection)와 같은 기술적 문제 해결을 목적으로 한 데이터가 아니다. 그것은 **인문**

학적 이해를 기반한 전자적 기술 체계를 선결 조건으로 요하며, 그 결과 기술 환경에서 새로운 인문학적 연구 및 활용을 가능케 하는 데이터이다.

인문 레이블 데이터와 파인튜닝

현재 서비스 중인 ChatGPT를 위시한 사전 훈련된(pre-trained) AI 모델은 코딩이나 업무 처리에도 동원되는 등 상당히 우수한 성능을 보이고 있지만, 당연하게도 인문학이라는 도메인에서 다루는 역사적 객체 등에 대해서는 대부분의 경우 제대로 인식하지 못한다. 이는 모델 학습에 활용된 기존의 학습 데이터셋에 해당 도메인과 관련된 객체 정보가 부재하기 때문이다.

그렇기 때문에 딥러닝에 기반한 인문학 연구를 성공적으로 수행하기 위해서는, **연구 도메인에 맞는 자체적인 파인튜닝(fine-tuning)용 인문 데이터를 직접 구축**할 필요가 있다. 여기서 '**파인튜닝(fine-tuning)**'이란, 사전 학습된 모델의 매개변수(parameter)가 새로운 데이터로 학습되는, 전이 학습(transfer learning) 접근을 말한다.

개인이 감당할 수 없을 정도로 방대한 데이터로 훈련된 사전 학습 모델이 이미 숱하게 공개되어 있는 지금, 합목적적으로 설계, 구축된 인문 데이터셋은 사전 학습 모델을 특정 분야에 전문화시키기 위한 열쇠가 된다. 이는 비유하자면, 공교육을 거치며 기초적인 학습 능력을 배양한 학생이 대학과 대학원의 전공 학습을 통해 전문가에 가까워지는 것과 유사하다. 이렇게 특정 분야에 전문화된 모델은 연구자가 요구하는 수준의, 구체적이고 일관적인 작업이 가능해진다.

상기 이미지는 한국고전번역원에서 서비스 중인 한문고전 자동번역 베타 서비스의 사례이다. 이와 같은 서비스는 기구축된 양질의 번역 데이터 없이는 당초에 불가능하며, 목적에 맞는 적절한 데이터셋의 유무가 인문학적으로 의미 있는 모델 구현을 좌우함을 알 수 있다.

그럼 다음 절에서, 본격적인 인문 데이터셋을 모델에 파인튜닝하는 과정을 살펴보도록 한다.

2) 인문 레이블 데이터 파인튜닝

이 절에서는 파인튜닝이 무엇이며, 어떤 과정을 거쳐 파인튜닝을 수행할 수 있는지 살펴보도록 한다.

인문 데이터를 활용한 사전 학습 모델 파인튜닝 과정

사전 학습 모델(Pre-trained Model)을 '**미세조정(Fine-tuning)**'하는 것은 방대한 양의 텍스트(학습 데이터)를 통해 이미 학습을 마친 AI 모델을, 특정 분야의 데이터로 추가 학습을 시켜 해당 도메인에 전문화시키는 조정 과정이라 할 수 있다. 이는 마치 중학교, 고등학교를 졸업한 학생이 대학, 나아가 대학원에 진학해 특정 전공 분야를 추가로 학습하면서 해당 분야에 대한 전문가로 거듭나는 것에 비유할 수 있다.

- **파인튜닝 실습 코드**

상기 링크를 통해 제공하는 코드 `finetuning_kcelectra_kote_kpoem.ipynb`는 이러한 과정을 잘 보여주는 예시로, 실제 인문 레이블 데이터를 사전 학습 모델에 파인튜닝하기 위해 작성된 소스 코드이다.

1. **기반 모델(Base Model):** 우선 `beomi/KcELECTRA-base`와 같이 한국어에 대한 전반적인 이해 능력을 갖춘 사전 학습 모델을 토대로 사용한다. 해당 모델은 BERT 계열의 ELECTRA 모델을 한국어 데이터셋으로 학습시킨 사전 학습 모델로, 한국어의 보편적 구조를 이미 파악하고 있기에 밑바닥부터 학습시키는 고생을 덜어 준다.
2. **인문 레이블 데이터(Labeled humanities data):** 앞서 구축 사례로 들었던 `KPoEM` 데이터셋과 같이, 인문학 연구 목적을 위해 전문가가 직접 라벨을 부착한 인문 데이터를 활용해야 한다. 이러한 데이터는 모델이 배워야 할 특정 과업을 담고 있는데, `KPoEM` 데이터셋의 경우, 한국 근대 시 텍스트의 감정 식별에 특화되어 있으므로, '한국 근대 시 텍스트에서 감정 유형을 식별하고 분류하는 방법'을 내포하고 있다.
3. **미세 조정(Fine-tuning):** 기반 모델에 특정 도메인의 데이터를 추가로 학습시킨다. 이 과정을 거치며 모델의 매개변수(파라미터), 가중치가 특정 작업에 맞게 미세하게 조정된다. `KPoEM`으로 학습시키는 경우, 모델은 한국 근대시에 담긴 복합적인 감정을 분류하는 데 특화된 능력을 갖추게 된다.

이러한 접근 방식은 개인이 감당하기 어려울 정도로 막대한 비용과 시간이 드는, 언어 모델을 처음부터 학습시키는 방법 대신, 상대적으로 적은 데이터와 자원으로 특정 분야에서 높은

성능의 모델을 만들 수 있게 하는 효율적인 방법론으로 널리 활용되고 있다.

KPoEM 데이터를 활용하여 두 가지 주요 방식으로 한국 근대시 감정 분석 모델을 구축하였다.

1. **직접 미세조정 (Direct Fine-tuning):** 범용 한국어 모델(KcELECTRA)을 KPoEM 데이터셋에 직접 학습시켜 시(詩) 감성 분석에 특화된 '모델 A'를 구축하는 방식이다. 이 모델은 시의 독특한 언어 구조와 감성 표현을 집중적으로 학습한다.
2. **순차적 미세조정 (Sequential Fine-tuning):** 더 큰 규모의 일반적인 데이터셋(KOTE)으로 1차 학습을 진행한 후, 그 모델을 다시 고품질의 KPoEM 데이터셋으로 2차 학습시켜 '모델 B'를 구축하는 방식이다. 이는 범용적인 언어 이해 능력을 먼저 갖춘 뒤, 특정 도메인(시)에 대한 전문성을 심화시키는 전략이다. 코드의 최종 평가 결과에 따르면, 이 순차적 미세조정 방식이 가장 높은 성능을 보였다.

파인튜닝 상세 과정

인문 레이블 데이터를 사전 학습 모델에 파인튜닝하는 과정은 일반적으로 다음과 같은 단계로 구성된다.

1. 기본 설정 및 환경 구축

미세조정을 시작하기 전, 필요한 라이브러리를 설치하고 실험의 재현성을 보장하기 위한 환경을 설정하는 단계이다.

- **라이브러리 임포트:** `transformers`, `pytorch_lightning`, `pandas` 등 모델링, 학습, 데이터 처리에 필요한 도구들을 불러온다.
- **환경 설정:** 실험 결과를 일관되게 얻기 위해 랜덤 시드(Random Seed)를 고정한다. 또한, 데이터와 학습된 모델이 저장될 디렉토리를 설정한다.

2. 데이터 준비

모델이 학습할 수 있는 형태로 원시 데이터를 가공하고 준비하는 핵심적인 단계이다.

- **데이터 로딩:** `.tsv`, `.csv` 등 다양한 형태의 인문 레이블 데이터를 불러온다. 예시 코드에서는 `pandas`를 사용해 `KPoEM`과 `KOTE` 데이터셋을 로드했다.
- **데이터 전처리:** 이 단계는 데이터의 특성에 따라 가장 많은 노력이 필요할 수 있다.
  - **라벨 변환:** '슬픔', '기쁨'과 같은 텍스트 라벨이나 `"[5, 8]"` 같은 문자열 형식의 라벨을 모델이 이해할 수 있는 숫자 벡터(예: One-hot vector `[0, 0, ..., 1, ...]`)로 변환한다. 예시 코드의 `preprocess_kote` 함수가 이 역할을 수행한다.
  - **주석자(Annotator) 데이터 통합:** 여러 명의 사람이 라벨을 단 경우, 이들의 의견을 하나로 통합하는 과정이 필요하다. 예시 코드의 `preprocess_by_paper_method` 함수는 5

명의 주석자가 단 라벨의 등장 횟수를 계산하고, 이를 정규화한 뒤 특정 임계값을 기준으로 최종 라벨을 결정한다.
    - **텍스트 정제:** 분석에 불필요한 노이즈(특수문자, 오탈자 등)를 제거하는 작업을 포함할 수 있다.
- **데이터 분할:** 전체 데이터를 학습(Training), 검증(Validation), 테스트(Test)용으로 분할한다.
    - **학습 데이터:** 모델을 직접 학습시키는 데 사용된다.
    - **검증 데이터:** 학습 중인 모델의 성능을 중간 점검하여 과적합(Overfitting) 여부를 판단하고, 최적의 하이퍼파라미터를 찾는 기준이 된다.
    - **테스트 데이터:** 학습과 검증 과정에 전혀 사용되지 않은 데이터로, 최종 완성된 모델의 객관적인 성능을 평가하는 데 사용된다.

3. 모델 파이프라인 구축

데이터를 모델에 효율적으로 공급하고, 모델의 구조를 정의하며, 학습 과정을 설계하는 단계이다. `PyTorch Lightning`과 같은 프레임워크를 사용하면 이 과정이 매우 체계화된다.

- **토크나이저(Tokenizer) 로딩:** 사전 학습 모델에 맞춰 텍스트를 숫자 시퀀스(토큰)로 변환하는 토크나이저를 불러온다. 문장의 길이를 맞추기 위한 `padding`, 최대 길이를 넘는 부분을 자르는 `truncation` 등의 옵션을 설정한다.
- **데이터셋(Dataset) 클래스 정의:** Pytorch의 `Dataset` 클래스를 상속받아, 데이터 하나하나를 어떻게 가져와서 토큰화하고 라벨과 함께 묶을지 정의한다. 예시의 `PoetryDataset`은 텍스트와 라벨 벡터를 받아 토크나이저를 적용한 후 텐서(tensor) 형태로 반환한다.
- **데이터 모듈(DataModule) 정의:** `Pytorch Lightning`의 `DataModule`은 데이터셋을 받아 배치(batch) 단위로 묶어주는 `DataLoader`를 생성하는 역할을 표준화한다. 이를 통해 학습, 검증, 테스트 데이터 공급 로직을 학습 코드와 분리하여 관리가 용이해진다.
- **모델 아키텍처(Model Architecture) 정의:** `Pytorch Lightning`의 `LightningModule`을 상속받아 모델의 구조와 학습 로직을 정의한다. 예시의 `BaseTagger`가 이에 해당한다.
    - 사전 학습된 `ELECTRA` 모델을 불러온다.
    - 그 위에 새로운 분류기(Classifier, 주로 `nn.Linear` 레이어)를 추가한다. 이 분류기의 출력 크기는 예측해야 할 라벨의 총 개수와 같다.
    - 손실 함수(Loss Function)를 정의한다(예: 다중 라벨 분류를 위한 `BCELoss`).
    - 최적화 알고리즘(Optimizer, 예: `AdamW`)과 학습률 스케줄러(Learning Rate Scheduler)를 설정한다.

4. 모델 학습 (미세조정)

준비된 데이터와 모델 파이프라인을 사용하여 실제로 모델을 학습시키는 단계이다.

- **학습 실행:** `PyTorch Lightning`의 `Trainer` 객체는 이 과정을 자동화한다. `trainer.fit(model, datamodule)` 한 줄로 GPU 사용, 에포크(Epoch) 반복, 로깅 등 복잡한 학습 루프가 실행된다.
- **모델 저장:** `ModelCheckpoint` 콜백(callback)을 사용하여 검증 데이터셋에서 가장 좋은 성능(예: 가장 낮은 손실 값)을 보인 시점의 모델 가중치를 파일로 저장한다.

Optuna를 활용한 하이퍼파라미터 튜닝

모델의 성능은 학습률(learning rate), 배치 크기(batch size), 드롭아웃 비율(dropout rate) 등 개발자가 직접 설정해야 하는 **하이퍼파라미터(Hyperparameter)** 에 크게 좌우된다. **Optuna**는 이러한 최적의 하이퍼파라미터 조합을 체계적이고 자동화된 방식으로 찾아주는 라이브러리이다.

Optuna의 역할과 과정

Optuna는 '최적화 연구(study)'를 통해 지정된 범위 내에서 다양한 하이퍼파라미터 조합을 시도하고, 어떤 조합이 가장 좋은 성능을 내는지 탐색한다.

1. **Objective 함수 정의:** 최적화할 목표를 정의하는 함수(예: 코드의 `objective_kpoem`)를 작성한다. 이 함수는 Optuna의 `trial` 객체를 인자로 받는다.
    - **탐색 공간 정의:** `trial` 객체의 `suggest_float`, `suggest_categorical` 등의 메서드를 사용해 각 하이퍼파라미터가 탐색할 값의 범위나 목록을 지정한다. (예: 학습률은 `1e-6`에서 `5e-5` 사이의 로그 스케일 값)
    - **모델 학습 및 평가:** `trial`이 제안한 하이퍼파라미터 조합으로 모델을 생성하고, 검증 데이터셋으로 짧게(예: 2~3 에포크) 학습시킨다.
    - **성능 지표 반환:** 함수의 마지막에 최적화의 기준이 될 성능 지표(예: `validation_loss`)를 반환한다. Optuna는 이 반환 값을 최소화(또는 최대화)하는 것을 목표로 삼는다.
2. **Study 생성 및 최적화 실행:**
    - `optuna.create_study(direction='minimize')` 코드로 '연구' 객체를 생성한다. `direction` 인자는 objective 함수가 반환하는 값을 최소화할지, 최대화할지를 결정한다.

- `study.optimize(objective_fn, n_trials=15)` 코드로 지정된 횟수(`n_trials`)만큼 objective 함수를 반복 실행하여 최적의 조합을 탐색한다.

3. **최적 파라미터 활용:**
   - 탐색이 완료되면 `study.best_params`를 통해 가장 좋은 성능을 보였던 하이퍼파라미터 조합을 얻을 수 있다.
   - 이 최적의 조합을 사용하여 최종 모델을 더 긴 에포크 동안 안정적으로 학습시켜 최고의 성능을 이끌어낸다.

이처럼 Optuna를 사용하면 개발자의 직감에 의존한 수동적인 튜닝 과정을 자동화하여, 더 객관적이고 효율적으로 모델의 잠재 성능을 최대로 끌어낼 수 있다.

3) 구현 모델의 성능 평가

성능 평가는 학습이 완료된 모델이 마주하는 최종 시험이다. 이 단계는 학습 과정에 전혀 사용되지 않은 새로운 데이터(테스트 데이터셋)에 대해 모델이 얼마나 뛰어난 성능을 보이는지 객관적인 지표로 측정하는 과정이다. 이를 통해 모델이 단순히 학습 데이터를 암기한 것이 아니라, 새로운 데이터에 대해서도 일반화된 성능을 갖추었음을 증명한다.

평가 과정의 코드 단위 설명

제공된 코드의 `evaluate_model` 함수는 평가 과정을 체계적으로 보여준다.

1. **모델 준비:** `model.to(device)` 코드를 통해 평가에 사용할 GPU(또는 CPU) 장치에 모델을 올린다. 이후 `model.eval()`을 호출하여 모델을 '평가 모드'로 전환한다. 이 모드에서는 학습 시에만 필요한 드롭아웃(Dropout)이나 배치 정규화(Batch Normalization) 등의 기능이 비활성화되어 일관된 예측 결과를 얻을 수 있다.
2. **예측 생성:** `for item in tqdm(test_dataset, ...)` 루프를 통해 테스트 데이터셋의 모든 데이터를 하나씩 모델에 입력한다. `with torch.no_grad():` 구문은 예측 시 불필요한 기울기 계산을 방지하여 메모리 사용량을 줄이고 계산 속도를 높이는 역할을 한다. `_, preds = model(input_ids, attention_mask)` 코드는 모델로부터 각 라벨에 대한 확률 값을 포함한 예측 결과(`preds`)를 생성한다.
3. **결과 취합 및 변환:** 모델이 생성한 예측 결과와 실제 정답 라벨을 `predictions_list`와 `labels_list`에 순서대로 쌓는다. 루프가 끝나면 `torch.stack`을 사용해 이 리스트들을 하나의 큰 텐서(Tensor)로 결합한다. 마지막으로, `(predictions_np > threshold).astype(int)` 코드를 통해 모델이 출력한 확률 값을 0 또는 1의 최종 예측 값으로 변환한다. 예를 들어, 임계값(threshold)이 0.3일 때, 0.5의 확률 값은 1(해당 라벨이 있음)로, 0.2의 확률 값은 0(해당 라벨이 없음)으로 변환된다.

코드에 사용된 주요 성능 평가지표

채점에는 다양한 지표가 사용되며, 각 지표는 모델의 특정 성능 측면을 보여준다.

ACTUAL VALUES	POSITIVE	NEGATIVE
POSITIVE	TP	FN
NEGATIVE	FP	TN

$$Precision = \frac{TP}{TP + FP} \qquad Recall = \frac{TP}{TP + FN}$$

$$Accuracy = \frac{TP + TN}{TP + FP + FN + TN}$$

$$F1\ Score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

출처: 10.3390/electronics12030585

- **정확도 (Accuracy):** 가장 직관적인 지표로, 전체 예측 중 정답을 맞힌 비율이다. `torchmetrics.functional.classification.multilabel_accuracy` 함수를 통해 계산된다. 그러나 데이터의 라벨 분포가 불균일할 경우, 다수 라벨에 대한 성능만으로도 지표가 높게 나타나는 착시를 일으킬 수 있다.
- **정밀도 (Precision):** 모델이 특정 라벨(예: '분노')이라고 예측한 결과물 중에서, 실제로 정답이 '분노'였던 것의 비율이다. 이 지표는 "모델의 예측을 얼마나 신뢰할 수 있는가?"에 대한 답을 준다. `sklearn.metrics.precision_score`로 계산할 수 있다.
- **재현율 (Recall):** 실제 '분노' 라벨을 가진 데이터 중에서, 모델이 '분노'라고 올바르게 예측한 것의 비율이다. 이 지표는 "모델이 정답을 놓치지 않고 얼마나 잘 찾아내는가?"를 나타낸다. `sklearn.metrics.recall_score`로 계산된다.
- **F1 점수 (F1-Score):** 정밀도와 재현율의 조화 평균이다. 두 지표가 모두 중요한 상황에서 이 둘의 균형을 고려한 단일 점수를 제공하므로 모델의 종합 성능을 평가하는 데 널리 사용된다. `sklearn.metrics.f1_score` 함수로 계산한다.
- **매튜스 상관 계수 (MCC, Matthews Correlation Coefficient):** 모델의 예측과 실제 정답 사이의 상관관계를 -1에서 +1 사이의 값으로 나타내는 지표이다. 데이터 불균형 상황에서도 다른 지표에 비해 안정적인 성능 측정이 가능하여 신뢰도가 높다. +1은 완벽한 예측, 0은 무작위 예측 수준을 의미한다. `sklearn.metrics.matthews_corrcoef`로 계산한다.
- **Micro & Macro 평균:** 정밀도, 재현율, F1 점수를 다중 라벨 환경에서 계산할 때 사용되는 평균 방식이다.
    - **Macro 평균:** 모든 라벨 각각의 성능 지표를 개별적으로 계산한 후, 이들의 산술 평균을 낸다. 희귀 라벨의 성능을 동등하게 반영하여, 모델이 소수 라벨을 얼마나 잘 처리하는지 확인하기에 좋다.

- **Micro 평균:** 모든 라벨의 예측 결과를 하나로 합산하여 전체적인 성능 지표를 계산한다. 데이터 수가 많은 라벨의 성능에 더 큰 가중치를 두는 효과가 있다.

4) KPoEM 모델 활용과 의의

KPoEM(Korean Poetry Emotion Mapping) 데이터셋을 기반으로 구축된 모델은 기존의 감정 분석 데이터셋과는 그 목적이 다르다. KPoEM 데이터셋에 내재된 한국 근대시의 고유한 감정을 기계적 형식으로 변환하는 데 그 목적이 있다. 이렇게 구축된 모델은 그렇다면 어떻게 활용할 수 있고, 어떤 의의를 가지는가?

KPoEM 데이터셋의 특징

KPoEM은 한국 시(Poetry)를 행(line)과 작품(poem) 단위로 나누고, KOTE 데이터셋에서 설정한 44개의 세분화된 감정 라벨을 부착한 데이터셋이다.

KPoEM 데이터셋의 가장 큰 특징은 앞서 살펴봤듯이, 시어(詩語)가 가진 중의성, 은유, 그리고 복합적인 감정선을 다룬다는 점이다. 이는 기존의 감정 데이터셋이 대개 온라인 댓글 등의 텍스트에 초점을 맞춘 것과는 차이를 보인다.

또한, 5명의 라벨러(annotator)가 평가한 라벨 정보를 모두 포함하고 있어, 라벨 간의 일치도를 바탕으로 감정 간 합치의 정도를 정량화할 수 있는 고품질 데이터이다. 파인튜닝을 위해 활용한 코드에서는 여러 라벨러의 라벨 등장 횟수를 min-max로 점수화하고, 이를 정규화하여 최종 학습 라벨을 생성하는 방식으로 평가자 간 신뢰도(Inter-rater Reliability)를 제고하였다.

KPoEM 모델의 활용 방안

이렇게 구축된 KPoEM 모델은 다음과 같은 분야에서 활용될 수 있다.

- **문학 연구 및 분석:** 특정 시대나 작가의 작품에 나타나는 감성 흐름, 주요 감성 등을 계량적으로 분석하여 문학 연구의 깊이를 더하는 보조 도구로 사용될 수 있다.
- **교육용 AI 보조교사:** 학생들이 시를 읽고 그 안에 담긴 복합적인 감정을 이해하고 공감할 수 있도록 돕는 학습 보조 도구를 개발하는 데 활용될 수 있다.
- **AI 창작 및 스토리텔링:** 특정 감성이나 분위기를 담은 시 또는 문학 텍스트를 생성하는 AI 모델의 기반 기술로 응용될 수 있다.

KPoEM 모델의 의의

KPoEM 모델의 개발과 활용은 다음과 같은 중요한 의의를 가진다.

첫째, **컴퓨터가 인간의 문학적 감성을 이해하는 가능성을 제시한다.** 긍정/부정과 같은 단순한 감정 분석을 넘어, '안타까움/실망', '우쭐댐/무시함', '비장함'과 같이 복잡하고 미묘한 인간의 감정을 언어 모델이 학습하고 분류할 수 있음을 보여준다.

둘째, **고품질 데이터셋의 중요성을 증명한다.** 코드의 실험 결과는 대규모의 일반 데이터(KOTE)로 학습한 모델보다, 특정 도메인에 맞춰 정교하게 설계된 고품질 데이터(KPoEM)로 최종 학습한 모델의 성능이 더 우수함을 명확히 보여준다. 이는 AI 모델의 성능이 데이터의 양뿐만 아니라 질에 크게 좌우된다는 점을 시사한다.

셋째, **인문학과 기술의 융합을 통한 새로운 가치 창출이다.** 문학이라는 인문학의 정수를 데이터화하고, 이를 자연어 처리 기술로 분석함으로써 새로운 방식의 학술 연구와 문화 콘텐츠 창출의 길을 연다. 이는 기술이 인문학적 소양을 깊게 하고, 인문학이 기술에 새로운 도전 과제를 제시하는 선순환 구조를 만드는 중요한 사례이다.